

1.- **[Obligatorio]** Hacer un análisis del problema, plantear la ecuación de recurrencia correspondiente y proponer un algoritmo por programación dinámica que obtenga la solución óptima al problema planteado: Dada la secuencia de energía disponible desde el último descanso $s_1, s_2, \dots, s_n$, y el esfuerzo/ganancia de cada día e_i , determinar la máxima cantidad de ganancia que se obtiene de los entrenamientos, considerando posibles descansos.

El problema planteado pone a Scaloni en una posición en la que debe decidir cuál es la forma de hacer que su equipo entrene de manera óptima. Se tiene una cantidad determinada de días a entrenar, una cantidad de energía de los jugadores que va disminuyendo a lo largo que entrenan días seguidos y por parámetro se obtiene una ganancia por día entrenado, esta ganancia se define entre el mínimo valor que exista entre la ganancia predeterminada y la energía que los jugadores tengan al momento de entrenar, lo que en otras palabras quiere decir que:

$$\text{Ganancia de un día entrenado} = \min(\text{Energía Actual}, \text{Ganancia del día})$$

Como opción de toma de decisión se tiene la posibilidad de hacer descansar a los jugadores, esto con el fin de reiniciar su energía, empezando otra vez con la misma energía del primer día, descansar un día implica también que los jugadores no pueden entrenar y por ende no poseen ganancia de entrenamiento ese día.

Para resolver este problema se plantea la siguiente ecuación de recurrencia:

$$EO[i][j] = \max(\min(S[j], E[i]) + EO[i + 1][j + 1], EO[i + 1][1])$$

EO = representa la máxima ganancia posible desde el día i hasta el día n,

habiendo entrenado j días consecutivos desde el último descanso.

i = día actual

j = cantidad de días consecutivos entrenados desde el último descanso ($1 \leq j \leq n$).

E_i = ganancia preestablecida del día actual

S_j = Energía que poseen los jugadores el día evaluado

La cantidad de días a evaluar es de n días. por ende la ecuación de recurrencia está definida en un rango de 1 a n inclusive, lo que significa que $EO[n+1][j] = 0$, para todo j mayor que 0.

El algoritmo propuesto recibe una lista de entrenamientos que posee las ganancias por entrenamientos, y a su vez recibe una lista de energías de los jugadores, evalúa en base a la ecuación de recurrencia cuales son los días de entrenamiento óptimo y devuelve una lista indicando que días entrenar y descansar, dejando así como resultado un orden de entrenamiento óptimo y ya definido.

2.- **[Obligatorio]** Demostrar que la ecuación de recurrencia planteada en efecto nos permite obtener la solución óptima al problema.

La ecuación de recurrencia definida con anterioridad depende principalmente de los días que llevamos sin entrenar y el día actual de entrenamiento. Al ser planteada como una solución "Bottom-up" se almacenan los resultados de subproblemas en una tabla, evitando recursividad.

En cada estado se consideran todas las decisiones posibles (entrenar o descansar) y se selecciona aquella que maximiza la ganancia total.

Esta ecuación da una solución que se es válida debido al principio de optimalidad: toda solución óptima está compuesta por soluciones óptimas de sus subproblemas. Además, se consideran todas las decisiones posibles en cada etapa y se selecciona la mejor.

DEMOSTRACIÓN POR CONTRADICCIÓN

Retomando la ecuación de recurrencia:

$$EO[i][j] = \max(\min(E_i, S_j) + EO[i+1][j+1], EO[i+1][1])$$

Si intentamos demostrar por contradicción, entonces en un estado $[i][j]$, $EO[i][j]$ (la ecuación de recurrencia planteada) no da el resultado óptimo, a este resultado no óptimo lo llamaremos $A1$. Se supone $A2$ la solución óptima al mismo estado.

Al ser $A2$ óptima tiene más ganancia que $A1$, es decir $A2 > A1$

A2 es óptima y nos deja su evaluación en dos casos bases:

Caso 1: consiste en entrenar el día evaluado. lo que consta de verificar el mínimo entre la energía y el día actuales, a ese resultado sumarle los días óptimos de los siguientes entrenamientos, es decir que cumple con:

$$OPT = \min(S[j], E[i]) + OPT[i+1][j+1]$$

Por hipótesis inductiva, asumimos que los subproblemas están correctamente resueltos, es decir:

$$OPT[i+1][j+1] = EO[i+1][j+1]$$

$$\text{Entonces: } OPT[i][j] = \min(S[j], E[i]) + EO[i+1][j+1]$$

Pero por definición de la recurrencia:

$$EO[i][j] \geq \min(S[j], E[i]) + EO[i+1][j+1]$$

Por lo tanto:

$$EO[i][j] \geq OPT[i][j]$$

Lo cual contradice que $A2 > A1$.

Caso 2: Propone descansar el día evaluado, lo que implica no sumar ganancia ese día pero aumentar la energía de los jugadores, permitiendo la posibilidad de tener mejor ganancia en entrenamientos futuros. es decir que cumple con:

$$OPT = OPT[i+1][0]$$

Nuevamente, por hipótesis inductiva:

$$OPT[i+1][1] = EO[i+1][1]$$

$$\text{Entonces: } OPT[i][j] = EO[i+1][1]$$

Y por la recurrencia:

$$EO[i][j] \geq EO[i+1][1]$$

$$\text{Por lo tanto: } EO[i][j] \geq OPT[i][j]$$

Lo cual contradice nuevamente que $A_2 > A_1$.

Ambos casos llevan a una contradicción, por lo tanto la suposición inicial ($A_2 > A_1$) es falsa. En consecuencia, la recurrencia calcula correctamente la solución óptima.